

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Software Requirements Specification

File Compression Tool (Basic ZIP Implementation)

Version 1.1 approved

Prepared by Team 14

Inchara K Kuppal (PES1UG25CS818)

Pavan Kumar HK (PES1UG24CS319)

Rachit Upadhyay (PES1UG24CS355)

Pranavi Sistla (PES1UG24CS338)

Organization: team14-file-compression

Date of Creation: October 4, 2026

PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Table of Contents

1. Introduction
    - 1.1 Purpose
    - 1.2 Intended Audience
    - 1.3 Product Scope
    - 1.4 References
2. Overall Description
    - 2.1 Product Perspective
    - 2.2 Product Functions
    - 2.3 User Classes and Characteristics
    - 2.4 Operating Environment
    - 2.5 Design and Implementation Constraints
    - 2.6 Assumptions and Dependencies
3. External Interface Requirements
    - 3.1 User Interfaces
    - 3.2 Software Interfaces
    - 3.3 Communications Interfaces
    - 3.4 Hardware Interfaces
4. Analysis Models
    - 4.1 UML Use Case Model
5. System Features
    - 5.1 System Feature 1: File Compression
    - 5.2 System Feature 2: File Decompression
6. Other Non-functional Requirements
    - 6.1 Performance Requirements
    - 6.2 Safety Requirements
    - 6.3 Security Requirements
    - 6.4 Software Quality Attributes
    - 6.5 Business Rules
7. Other Requirements (Plan of Work and Product Ownership)

- Appendix A: Glossary
- Appendix B: Field Layouts
- Appendix C: Requirement Traceability Matrix



PES UNIVERSITY, BANGALORE

Department of Computer Science and Engineering

Revision History

Name	Date	Reason For Changes	Version
Team 14	October 1, 2026	Initial draft creation.	1.0
Team 14	October 4, 2026	Added Analysis Models, Plan of Work, References, and RTM to comply with project guidelines.	1.1

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) details the requirements for the File Compression Tool (Basic ZIP Implementation). It outlines the system's functional and non-functional requirements, providing a clear roadmap for the developers and a comprehensive overview for evaluators. This document covers the entire scope of the standalone C/C++ application.

1.2 Intended Audience

This document is intended for project evaluators, software developers, and testers. It is organized logically to allow developers to understand the structural constraints (like the mandated C/C++ environment and Huffman coding), while providing project reviewers with the necessary context to assess functional completeness.

1.3 Product Scope

The software being specified is a basic ZIP compression tool that utilizes Huffman coding for file size reduction. Its primary objective is to allow users to efficiently compress large files to save disk space and decompress them losslessly. The project aligns with foundational data structure and algorithm implementation goals.

1.4 References

Project Guidelines: SE Mini Project Template and Evaluation Guidelines.

Algorithmic Foundation: Huffman, D. A. (1952). A Method for the Construction of Minimum-Redundancy Codes. *Proceedings of the IRE*, 40(9), 1098-1101. (Reference for core Huffman tree logic).

Format Specification: PKWARE Inc. (Standard). *ZIP File Format Specification*. (Reference for general archive structuring and metadata header concepts).

Language Standard: ISO/IEC 14882:2020 (C++20 Standard). Used for standard file I/O operations and memory management guidelines.

2. Overall Description

2.1 Product Perspective

This product is a new, self-contained desktop utility. It does not replace any existing commercial systems but serves as a fundamental implementation of compression algorithms. It is a standalone software component interacting directly with the host operating system's file management system.

2.2 Product Functions

- Read an input file of any standard format.
- Compress the file utilizing Huffman coding to reduce its overall size.
- Decompress a previously compressed file back to its exact original state (lossless).
- Display compression metrics (e.g., original size, compressed size, and compression ratio).

2.3 User Classes and Characteristics

General Users: Individuals looking for a lightweight, command-line-based utility to shrink file sizes for storage or transfer.

System Administrators/Developers: Users who require a fast, C/C++ based tool for batch processing or scripting environments.

2.4 Operating Environment

The software will operate on any standard operating system (Windows, Linux, macOS) that supports C/C++ compilation and execution. It requires minimal memory overhead and standard file system read/write permissions.

2.5 Design and Implementation Constraints

Language Requirement: The tool must be developed strictly in C/C++.

Algorithmic Constraint: The file size reduction must be achieved specifically using Huffman coding.

Documentation Constraint: The final project proposal must be submitted as a single PDF document by the given due date.

2.6 Assumptions and Dependencies

It is assumed that the host environment has standard C/C++ libraries available for file I/O operations.

The system depends on the host OS granting sufficient disk space and permissions to read the original file and write the new compressed/decompressed files.

3. External Interface Requirements

3.1 User Interfaces

The primary interface will be a Command Line Interface (CLI). The user will interact with the tool by passing arguments (e.g., file paths, flags for compression/decompression). Standard error messages will be displayed in the console for invalid inputs or file access errors.

3.2 Software Interfaces

The application will interact with the standard C/C++ input/output libraries (like <fstream> or <stdio.h>) to handle file reading and writing. No external third-party software databases or libraries are required.

3.3 Communications Interfaces

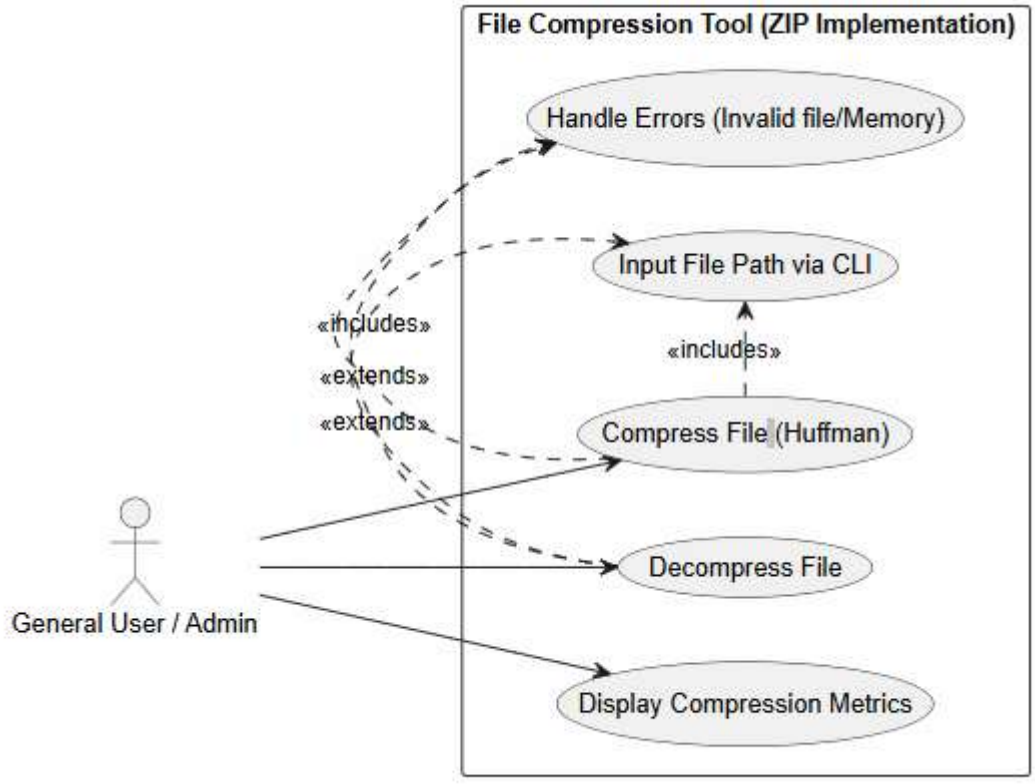
Not applicable. This product is a standalone local desktop utility and does not require network, email, or web browser communications protocols.

3.4 Hardware Interfaces

No specialized hardware interfaces are required beyond standard host system storage and memory access managed by the operating system.

4.Analysis Models

4.1 UML Use Case Model



5. System Features

5.1 System Feature 1: File Compression

5.1.1 Description and Priority

High priority. The core feature allowing the system to accept a file and compress it.

5.1.2 Stimulus/Response Sequences

The user inputs a command to compress a specific file. The system reads the file, generates a Huffman tree based on character frequency, encodes the data, and writes the compressed output to the disk.

5.1.3 Functional Requirements

REQ-1: The system shall accept a valid file path as input.

REQ-2: The system shall parse the file to calculate character frequencies.

REQ-3: The system shall construct a Huffman tree and generate encoded bitstreams.

REQ-4: The system shall output a compressed file with a custom extension (e.g., .huff or .zip).

5.2 System Feature 2: File Decompression

5.2.1 Description and Priority

High priority. The inverse operation to restore the file to its original state.

5.2.2 Stimulus/Response Sequences

The user provides the compressed file. The system reads the header/tree data, decodes the bitstream, and recreates the original file.

5.2.3 Functional Requirements

REQ-5: The system shall read the compressed file and extract the Huffman tree metadata.

REQ-6: The system shall decode the compressed bitstream using the extracted tree.

REQ-7: The system shall write the decompressed data to a new file, matching the exact original file size and contents.

6. Other Nonfunctional Requirements

6.1 Performance Requirements

The tool should compress and decompress text and binary files up to 100MB in a reasonable timeframe (under 10 seconds) on a standard modern processor. Memory consumption should not exceed the size of the file being processed plus tree overhead.

6.2 Safety Requirements

The system must safely handle file operations. If the disk is full or a file is corrupted, the program must terminate gracefully with a helpful error message rather than crashing or corrupting existing user data.

6.3 Security Requirements

As a local utility, standard OS file permissions apply. The tool will not bypass any system-level read/write security protocols.

6.4 Software Quality Attributes

Correctness: The compression must be strictly lossless; the decompressed file must be a byte-for-byte match of the original.

Portability: Written in standard C/C++, the source code must be easily compilable on different operating systems without heavy modification.

6.5 Business Rules

The software operates as an open, standalone academic utility without commercial role restrictions; any OS user with valid file read/write permissions may execute compression and decompression commands.

7. Other Requirements (Plan of Work and Product Ownership)

The short-term plan of work involves setting up the C/C++ build environment, establishing file I/O templates, and implementing the core Huffman tree logic over the next two weeks. Product ownership is distributed equally among team members, explicitly defining functional feature contributions and qualitative property responsibilities.

Team Ownership Matrix

- **Pranavi Sistla (PES1UG24CS338)**

Functional Features: Implement CLI input validation and file parsing to calculate character frequencies (REQ-1, REQ-2).

Qualitative Property: Ensure cross-platform portability across operating systems and validate CLI error handling.

- **Pavan Kumar HK (PES1UG24CS319)**

Functional Features: Construct the Huffman tree and generate encoded bitstreams (REQ-3).

Qualitative Property: Fine-tune system performance to ensure tree generation and encoding execute rapidly, achieving the response time requirement of under 10 seconds for 100MB files.

- **Rachit Upadhyay (PES1UG24CS355)**

Functional Features: Extract Huffman tree metadata from compressed files and decode bitstreams (REQ-5, REQ-6).

Qualitative Property: Ensure strict lossless correctness so that the decompressed output achieves a 100% byte-for-byte match with the original file.

- **Inchara K Kuppal (PES1UG25CS818)**

Functional Features: Handle file writing for compressed .huff archives and decompressed outputs, ensuring accurate sizing (REQ-4, REQ-7).

Qualitative Property: Optimize system memory management and dynamic allocation to ensure memory consumption stays strictly within the file size plus tree overhead limit.

Appendix A: Glossary

CLI: Command Line Interface.

Huffman Coding: A lossless data compression algorithm that assigns variable-length codes to input characters, with shorter codes assigned to more frequent characters.

ZIP: A common archive file format that supports lossless data compression.

Appendix B: Field Layouts

Binary .huff Archive Header & Payload Layout:

Field	Size / Length	Data Type	Description	Is Mandatory
Original File Size	8 Bytes	Unsigned 64-bit Integer (uint64_t)	Total byte count of the uncompressed input file	Y
Character Frequency / Tree Metadata	Variable / 256 Entries	Array of Unsigned Integers	Frequency table or serialized Huffman tree header used to reconstruct the tree	Y
Encoded Bitstream Payload	Variable	Binary Bitstream	Variable-length Huffman encoded data bytes	Y

Appendix C: Requirement Traceability Matrix

Sl. No	Requirement ID	Brief Description of Requirement	Architecture Reference	Design Reference	Code File Reference	Test Case ID	System Test Case ID
1	REQ-1	System shall accept a valid file path as input.	CLI Module	TBD	TBD	TC-COMP-01	STC-01
2	REQ-2	System shall parse the file to calculate character frequencies.	Compression Engine	TBD	TBD	TC-COMP-02	STC-01
3	REQ-3	System shall construct a Huffman tree and generate encoded bitstreams.	Compression Engine	TBD	TBD	TC-COMP-03	STC-01
4	REQ-4	System shall output a compressed file with a custom extension.	File I/O Module	TBD	TBD	TC-COMP-04	STC-01

Sl. No	Requirement ID	Brief Description of Requirement	Architecture Reference	Design Reference	Code File Reference	Test Case ID	System Test Case ID
5	REQ-5	System shall read compressed file and extract tree metadata.	Decompression Engine	TBD	TBD	TC-DECOMP-01	STC-02
6	REQ-6	System shall decode the compressed bitstream using the extracted tree.	Decompression Engine	TBD	TBD	TC-DECOMP-02	STC-02
7	REQ-7	System shall write decompressed data to a new file (lossless).	File I/O Module	TBD	TBD	TC-DECOMP-03	STC-02